

Cloud-Native Tactical RAG

7-Day Intensive Architecture Sprint · AWS, Docker, Airflow & Deep Learning

Lead Developer: Chintapenta Sai Srinivas

This document details a high-velocity, no-compromise 7-day development sprint. The resulting architecture integrates large-scale data engineering workflows with multimodal machine learning (Vision Transformers), representing a top-tier portfolio project optimized for MS application evaluations in Computer Science and AI programs.

Infrastructure & Topology Blueprint

AWS EC2

Docker

PostgreSQL / pgvector

The system operates on a containerized architecture deployed on an AWS EC2 instance (e.g., `g4dn.xlarge`) to support GPU-accelerated workloads.

- **Storage Engine:** A Dockerized PostgreSQL database running the `pgvector` extension for unified relational querying and high-dimensional semantic search.
- **Orchestration:** Apache Airflow clusters deployed via `docker-compose` (Scheduler, Webserver, Worker) to manage idempotent DAG execution.
- **Compute:** Dedicated Python containers to handle API serving (FastAPI) and Computer Vision tracking pipelines without blocking the web processes.

Days 1-2: Ingestion & Pipeline Orchestration

Apache Airflow

Python

Establishing the ETL foundation to systematically extract and normalize relational event data.

- Configure the `docker-compose.yml` file to spin up the Airflow and PostgreSQL/pgvector environments.
- Author the `extract_and_stage` DAG in Airflow to systematically fetch JSON match logs from the StatsBomb API.
- Implement transformation scripts using `pandas` to convert native 120x80 coordinates onto standard pitch dimensions, writing cleaned artifacts to a staging directory or S3 bucket.

Days 3-4: Vision Transformers & Homography

Deep Learning

OpenCV

The computationally intensive phase. Deriving real-time tactical spacing from unstructured video feeds.

- Ingest open-source broadcast footage and parse into frame sequences using OpenCV.
- Execute a Vision Transformer (ViT) architecture on the EC2 GPU to track individual players and extract precise bounding box coordinates per frame.
- Calculate the geometric homography matrix H to project 2D screen coordinates (u, v) onto the top-down 2D tactical pitch plane (X, Y) .
- Sync CV-derived positions with textual event logs, establishing a multimodal representation of the match phase.

Days 5-6: Vector Injection & RAG Synthesis

LLM Integration

Semantic Search

Building the cognitive layer by mapping spatial realities into language model context windows.

- Execute semantic chunking over textual scout reports and synthesize unified text prompts incorporating the normalized (X, Y) spatial coordinates.
- Route the fused data through an embedding model (e.g., text-embedding-3 or open-source equivalent).
- Load the resulting high-dimensional vectors directly into pgvector using bulk batch operations.
- Construct the RAG retrieval mechanism: map natural language queries to vector space, calculate cosine similarity, and retrieve the top K tactical scenarios.

Day 7: Full Stack Deployment & Visualization

FastAPI

Streamlit

Wiring the components into an interactive, portfolio-ready application.

- Finalize the FastAPI routing to expose endpoints that receive queries and return LLM-synthesized analytical responses.
- Deploy the Streamlit frontend.
- Integrate `mplsoccer` to dynamically render the digital pitch. When the LLM outputs a tactical summary, the system must simultaneously map the exact retrieved event locations visually onto the screen.

Implementation Note: This intensive schedule requires strict adherence to container isolation. Ensure all dependencies (Airflow workers, deep learning libraries, and web frameworks) are mapped correctly in the Docker volumes to prevent environment conflicts across the AWS EC2 instance.